Practice Lab 7 Solutions

Pointers, Memory Management & Dynamic Allocation

Programming with C++ (EEC-101)

Department of Electrical Engineering
Indian Institute of Technology Roorkee

Abstract

This document provides detailed theoretical explanations and complete C++ solutions for all 13 questions in Practice Lab 7. Topics covered include pointer mechanics, dereferencing, pointer arithmetic, generic pointers (`void*`), dynamic memory allocation (`new`/`delete`), array of function pointers, and 2D ragged arrays.

Compiled for Student Revision

Contents

1 Theoretical Fundamentals **2**
1.1 1. Pointers, Memory Addresses, and Dereferencing . . . 2
1.2 2. Pointer Arithmetic . . . 2
1.3 3. Pointers and Multidimensional Arrays . . . 2
1.4 4. Generic Pointers (`void*`) & Pointer Sizes . . . 2
1.5 5. Dynamic Memory Allocation (`new` / `delete`) . . . 2
1.6 6. Avoiding Dangling Pointers . . . 2

2 Solutions to Lab Questions **3**
2.1 Question 1: Variable Initialization and Modification via Pointer . . . 3
2.2 Question 2: Array Sum Function Returning a Safe Pointer . . . 3
2.3 Question 3: 2D Array Element & Address Display . . . 5
2.4 Question 4: Generic `void*` Pointer & Pointer Sizes . . . 5
2.5 Question 5: String Length & Concatenation Using Pointers . . . 7
2.6 Question 6: Element-Wise Array Addition via Dynamic Pointer . . . 7
2.7 Question 7: Array Traversal Modes via Pointer Arithmetic . . . 9
2.8 Question 8: Vowel Counter using Character Pointers . . . 9
2.9 Question 9: Array of Pointers for Max/Min Transformation . . . 11
2.10 Question 10: Dynamic Resizing 1D Array (`append` and `display`) . . . 11
2.11 Question 11: Array of Function Pointers . . . 13
2.12 Question 12: Dynamic 2D Ragged Array for Temperature Tracking . . . 13
2.13 Question 13: 2D Character Array Word Order and Addresses . . . 15

Theoretical Fundamentals

1. Pointers, Memory Addresses, and Dereferencing

Every variable in C++ resides at a unique contiguous memory address in RAM. The address-of operator (`&`) yields this hexadecimal address. A **pointer** is a specialized variable that stores the memory address of another variable.

- **Declaration & Assignment**: `int* p = &a;` binds pointer `p` to the address of `a`.
- **Dereferencing (`*`)**: Placing `*` before a pointer accesses or modifies the value stored at that address (`*p = 20;`).

2. Pointer Arithmetic

Pointer arithmetic is scaled by the size of the underlying data type (`sizeof(type)`):

- `ptr + i` calculates the memory address shifted by $i \times$ sizeof(type) bytes.
- `ptr1 - ptr2` yields the element distance (offset) between two pointers within the same array.

3. Pointers and Multidimensional Arrays

An array name acts as a constant pointer to its first element (`arr == &arr[0]`).

- **1D Array**: `arr[i]` is mathematically equivalent to `*(arr + i)`.
- **2D Array**: For `int arr[R][C]`, `*(arr + i) + j` yields the address of `arr[i][j]`, and `*(*(arr + i) + j)` dereferences its value.

4. Generic Pointers (`void*`) & Pointer Sizes

A `void*` is a generic pointer capable of storing addresses of any data type. It cannot be dereferenced directly without typecasting. On a given system architecture, all pointers occupy identical memory sizes (e.g., 8 bytes on 64-bit systems), regardless of the data type pointed to.

5. Dynamic Memory Allocation (`new` / `delete`)

Heap memory enables dynamic allocation when sizes are determined at runtime:

- `int* arr = new int[n];` allocates contiguous heap memory.
- `delete[] arr;` releases memory back to the OS to prevent memory leaks.
- **Ragged Arrays**: A 2D structure where rows possess varying column counts (`int** grid = new int*[rows];`).

6. Avoiding Dangling Pointers

Returning the address of a local stack variable causes undefined behavior (dangling pointer) because local stack frames are destroyed upon function exit. To return pointers safely, allocate memory on the heap (`new`) or use `static` duration.

Solutions to Lab Questions

Question 1: Variable Initialization and Modification via Pointer

Task: Declare two integer variables `a` and `b`, and an integer pointer `p`. Initialize and modify `a` and `b` using `p`, displaying values and addresses.

```cpp
#include <iostream>
using namespace std;

int main() {
    int a, b;
    int* p;

    // Initialize 'a' using pointer 'p'
    p = &a;
    *p = 10;

    // Initialize 'b' using pointer 'p'
    p = &b;
    *p = 20;

    cout << "--- Initial Values ---\n";
    cout << "a = " << a << " (Address &a = " << &a << ")\n";
    cout << "b = " << b << " (Address &b = " << &b << ")\n\n";

    // Modify values through pointer 'p'
    p = &a;
    *p = 100;

    p = &b;
    *p = 200;

    cout << "--- Modified Values via Pointer ---\n";
    cout << "a = " << a << "\n";
    cout << "b = " << b << "\n";

    return 0;
}
```

Question 2: Array Sum Function Returning a Safe Pointer

Task: Populate a 1D array with values 51 to 60 using pointer arithmetic. Implement `int* sum(int*, int)` returning a heap pointer to avoid dangling stack pointers.

```cpp
#include <iostream>
using namespace std;

int* sum(int* arr, int size) {
    int* total = new int(0); // Heap allocation prevents dangling pointer
    for (int i = 0; i < size; i++) {
        *total += *(arr + i); // Pointer arithmetic
    }
    return total;
}

int main() {
    int arr[10];
    int* ptr = arr;

    for (int i = 0; i < 10; i++) {
```

```
        *(ptr + i) = 51 + i;
    }

    cout << "Array Elements (51 to 60): ";
    for (int i = 0; i < 10; i++) {
        cout << *(ptr + i) << " ";
    }
    cout << "\n";

    int* sumPtr = sum(arr, 10);
    cout << "Sum of array elements = " << *sumPtr << endl;

    delete sumPtr; // Prevent memory leak
    return 0;
}
```

Question 3: 2D Array Element & Address Display

Task: Display elements and addresses of a 3×3 array using both subscript notation and pointer expressions.

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

    cout << "--- (i) Elements Without Pointers (Subscripts) ---\n";
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            cout << arr[i][j] << " ";
        }
        cout << "\n";
    }

    cout << "\n--- (ii) Elements Using Pointers (*(*(arr+i)+j)) ---\n";
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            cout << *(*(arr + i) + j) << " ";
        }
        cout << "\n";
    }

    cout << "\n--- Addresses Using Reference Operator (&arr[i][j]) ---\n";
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            cout << "&arr[" << i << "][" << j << "] = " << &arr[i][j] <<
                "\t";
        }
        cout << "\n";
    }

    cout << "\n--- Addresses Using Pointer Expressions (*(arr+i)+j) ---\n";
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            cout << "*(arr+" << i << ")+" << j << " = " << (void*)(*(arr +
                i) + j) << "\t";
        }
        cout << "\n";
    }

    return 0;
}
```

Question 4: Generic `void*` Pointer & Pointer Sizes

Task: Display addresses of `int`, `double`, and `char` arrays using a generic `void*` function. Check pointer size across types.

```cpp
#include <iostream>
#include <string>
using namespace std;

void printVoidAddresses(void* ptr, int count, int elementSize, const
    string& typeName) {
    cout << "Addresses of " << typeName << " array elements:\n";
    char* bytePtr = static_cast<char*>(ptr);
```

```
    for (int i = 0; i < count; i++) {
        void* addr = static_cast<void*>(bytePtr + i * elementSize);
        cout << "  Element [" << i << "]: " << addr << "\n";
    }
    cout << "\n";
}

int main() {
    int intArr[4] = {10, 20, 30, 40};
    double doubleArr[4] = {1.1, 2.2, 3.3, 4.4};
    char charArr[4] = {'A', 'B', 'C', 'D'};

    printVoidAddresses(intArr, 4, sizeof(int), "int");
    printVoidAddresses(doubleArr, 4, sizeof(double), "double");
    printVoidAddresses(charArr, 4, sizeof(char), "char");

    cout << "--- Pointer Sizes on This Architecture ---\n";
    cout << "sizeof(void*)    = " << sizeof(void*) << " bytes\n";
    cout << "sizeof(int*)     = " << sizeof(int*) << " bytes\n";
    cout << "sizeof(double*) = " << sizeof(double*) << " bytes\n";

    return 0;
}
```

Question 5: String Length & Concatenation Using Pointers

Task: Implement string length and string concatenation using pointer arithmetic without standard library string operations.

```cpp
#include <iostream>
using namespace std;

int getLength(const char* str) {
    int len = 0;
    while (*str != '\0') {
        len++;
        str++;
    }
    return len;
}

void concatenate(char* dest, const char* src) {
    while (*dest != '\0') {
        dest++;
    }
    while (*src != '\0') {
        *dest = *src;
        dest++;
        src++;
    }
    *dest = '\0';
}

int main() {
    char str1[100], str2[100], result[200];

    cout << "Enter first string: ";
    cin.getline(str1, 100);

    cout << "Enter second string: ";
    cin.getline(str2, 100);

    cout << "Length of String 1 = " << getLength(str1) << "\n";
    cout << "Length of String 2 = " << getLength(str2) << "\n";

    char* r = result;
    const char* s1 = str1;
    while (*s1 != '\0') {
        *r = *s1;
        r++; s1++;
    }
    *r = '\0';

    concatenate(result, str2);
    cout << "Concatenated String = " << result << "\n";

    return 0;
}
```

Question 6: Element-Wise Array Addition via Dynamic Pointer

Task: `int* addArrays(int* arr1, int* arr2, int size)` performs element-wise addition and returns a dynamically allocated array.

```cpp
#include <iostream>
```

```cpp
using namespace std;

int* addArrays(int* arr1, int* arr2, int size) {
    int* result = new int[size];
    for (int i = 0; i < size; i++) {
        *(result + i) = *(arr1 + i) + *(arr2 + i);
    }
    return result;
}

int main() {
    const int size = 5;
    int arr1[size] = {1, 2, 3, 4, 5};
    int arr2[size] = {10, 20, 30, 40, 50};

    int* sumArr = addArrays(arr1, arr2, size);

    cout << "Array 1: ";
    for (int i = 0; i < size; i++) cout << *(arr1 + i) << " ";
    cout << "\nArray 2: ";
    for (int i = 0; i < size; i++) cout << *(arr2 + i) << " ";

    cout << "\nSum Array: ";
    for (int i = 0; i < size; i++) cout << *(sumArr + i) << " ";
    cout << "\n";

    delete[] sumArr;
    return 0;
}
```

Question 7: Array Traversal Modes via Pointer Arithmetic

Task: Traversal modes using pointer operators (++, --, +, -): same order, reverse order, odd-indexed, and even-indexed elements.

```cpp
#include <iostream>
using namespace std;

void printSameOrder(const int* ptr, int size) {
    cout << "(i) Same order: ";
    for (int i = 0; i < size; i++) cout << *(ptr + i) << " ";
    cout << "\n";
}

void printReverseOrder(const int* ptr, int size) {
    cout << "(ii) Reverse order: ";
    const int* p = ptr + size - 1;
    for (int i = 0; i < size; i++) {
        cout << *p << " ";
        p--;
    }
    cout << "\n";
}

void printOddIndexed(const int* ptr, int size) {
    cout << "(iii) Odd-indexed elements (indices 0, 2, 4): ";
    const int* p = ptr;
    for (int i = 0; i < size; i += 2) {
        cout << *p << " ";
        p += 2;
    }
    cout << "\n";
}

void printEvenIndexed(const int* ptr, int size) {
    cout << "(iv) Even-indexed elements (indices 1, 3, 5): ";
    const int* p = ptr + 1;
    for (int i = 1; i < size; i += 2) {
        cout << *p << " ";
        p += 2;
    }
    cout << "\n";
}

int main() {
    int arr[6] = {10, 20, 30, 40, 50, 60};

    printSameOrder(arr, 6);
    printReverseOrder(arr, 6);
    printOddIndexed(arr, 6);
    printEvenIndexed(arr, 6);

    return 0;
}
```

Question 8: Vowel Counter using Character Pointers

Task: Count vowels in a string (max 19 characters) using character pointer traversal.

```cpp
#include <iostream>
#include <cctype>
```

```cpp
using namespace std;

int countVowels(const char* str) {
    int count = 0;
    while (*str != '\0') {
        char ch = tolower(*str);
        if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {
            count++;
        }
        str++;
    }
    return count;
}

int main() {
    char str[20];
    cout << "Enter a string (max 19 characters): ";
    cin.getline(str, 20);

    cout << "Number of vowels = " << countVowels(str) << endl;
    return 0;
}
```

Question 9: Array of Pointers for Max/Min Transformation

Task: Read 7 floats into an array, store element addresses in an array of pointers `float* ptrs[7]`, find max and min, double max and halve min.

```cpp
#include <iostream>
using namespace std;

int main() {
    float arr[7];
    float* ptrs[7];

    cout << "Enter 7 float numbers: ";
    for (int i = 0; i < 7; i++) {
        cin >> arr[i];
        ptrs[i] = &arr[i];
    }

    cout << "\nOriginal Numbers:\n";
    for (int i = 0; i < 7; i++) cout << *ptrs[i] << " ";
    cout << "\n\n";

    float** maxPtr = &ptrs[0];
    float** minPtr = &ptrs[0];

    for (int i = 1; i < 7; i++) {
        if (**ptrs[i] > **maxPtr) maxPtr = &ptrs[i];
        if (**ptrs[i] < **minPtr) minPtr = &ptrs[i];
    }

    cout << "Maximum Value: " << **maxPtr << "\n";
    cout << "Minimum Value: " << **minPtr << "\n\n";

    **maxPtr *= 2;
    **minPtr /= 2;

    cout << "After Modifying Maximum and Minimum:\n";
    for (int i = 0; i < 7; i++) cout << *ptrs[i] << " ";
    cout << "\n";

    return 0;
}
```

Question 10: Dynamic Resizing 1D Array (append and display)

Task: Implement dynamic array resizing that doubles capacity when full.

```cpp
#include <iostream>
using namespace std;

void append(int*& arr, int& size, int& capacity, int value) {
    if (size == capacity) {
        int newCapacity = capacity * 2;
        cout << "[Capacity reached (" << capacity << "). Doubling size to "
             << newCapacity << "...]\n";
        int* newArr = new int[newCapacity];
        for (int i = 0; i < size; i++) {
            newArr[i] = arr[i];
        }
        delete[] arr;
        arr = newArr;
```

```
        capacity = newCapacity;
    }
    arr[size] = value;
    size++;
}

void display(const int* arr, int size) {
    cout << "Array Elements: ";
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
    cout << "\n";
}

int main() {
    int capacity = 4;
    int size = 0;
    int* arr = new int[capacity];

    int testInputs[5] = {10, 20, 30, 40, 50};
    for (int i = 0; i < 5; i++) {
        cout << "Appending " << testInputs[i] << "...\n";
        append(arr, size, capacity, testInputs[i]);
    }

    cout << "\n--- Final Array State ---\n";
    display(arr, size);
    cout << "Current Capacity: " << capacity << "\n";

    delete[] arr;
    return 0;
}
```

Question 11: Array of Function Pointers

Task: Implement `square`, `cube`, `absoluteValue`, and `doubleValue`. Use an array of function pointers to execute mathematical sums over odd integers up to n.

```cpp
#include <iostream>
#include <string>
using namespace std;

int square(int x) { return x * x; }
int cube(int x) { return x * x * x; }
int absoluteValue(int x) { return x < 0 ? -x : x; }
int doubleValue(int x) { return x * 2; }

int sum(int (*func)(int), int n) {
    int total = 0;
    for (int i = 1; i <= n; i += 2) {
        total += func(i);
    }
    return total;
}

int main() {
    int (*operations[4])(int) = {square, cube, absoluteValue, doubleValue};
    string opNames[4] = {"Square", "Cube", "Absolute Value", "Double
       Value"};

    int n, choice;
    cout << "Enter integer n: ";
    cin >> n;

    cout << "Select Operation (0: Square, 1: Cube, 2: Absolute, 3: Double):
       ";
    cin >> choice;

    if (choice >= 0 && choice <= 3) {
        int result = sum(operations[choice], n);
        cout << "Sum of " << opNames[choice] << " for odd integers up to "
           << n << " = " << result << "\n";
    } else {
        cout << "Invalid choice!\n";
    }

    return 0;
}
```

Question 12: Dynamic 2D Ragged Array for Temperature Tracking

Task: Allocate a 2D ragged array for weekly temperature data, compute averages, global max/min, and deallocate memory.

```cpp
#include <iostream>
#include <climits>
using namespace std;

int main() {
    int numWeeks;
    cout << "Enter number of weeks: ";
    cin >> numWeeks;

    int* daysInWeek = new int[numWeeks];
```

```
    int** tempGrid = new int*[numWeeks];

    for (int i = 0; i < numWeeks; i++) {
        cout << "Enter number of days in week " << (i + 1) << ": ";
        cin >> daysInWeek[i];
        tempGrid[i] = new int[daysInWeek[i]];
    }

    for (int i = 0; i < numWeeks; i++) {
        cout << "Enter temperatures for week " << (i + 1) << ": ";
        for (int j = 0; j < daysInWeek[i]; j++) {
            cin >> tempGrid[i][j];
        }
    }

    cout << "\nAverage temperature per week:\n";
    int overallMax = INT_MIN;
    int overallMin = INT_MAX;

    for (int i = 0; i < numWeeks; i++) {
        int weekSum = 0;
        for (int j = 0; j < daysInWeek[i]; j++) {
            int t = tempGrid[i][j];
            weekSum += t;
            if (t > overallMax) overallMax = t;
            if (t < overallMin) overallMin = t;
        }
        int avg = weekSum / daysInWeek[i];
        cout << "Week " << (i + 1) << ": " << avg << "\n";
    }

    cout << "\nHighest temperature: " << overallMax << "\n";
    cout << "Lowest temperature: " << overallMin << "\n";

    for (int i = 0; i < numWeeks; i++) {
        delete[] tempGrid[i];
    }
    delete[] tempGrid;
    delete[] daysInWeek;

    return 0;
}
```

Question 13: 2D Character Array Word Order and Addresses

Task: Read 3 words into a 2D character array. Display words in order, in reverse order, and print the first character of each word alongside its memory address.

```cpp
#include <iostream>
using namespace std;

int main() {
    char words[3][10];

    cout << "Enter 3 words (max 9 characters each):\n";
    for (int i = 0; i < 3; i++) {
        cin >> words[i];
    }

    cout << "\nWords taken:\n";
    for (int i = 0; i < 3; i++) {
        cout << words[i] << "\n";
    }

    cout << "\nWords in Reverse Order:\n";
    for (int i = 2; i >= 0; i--) {
        cout << words[i] << "\n";
    }

    cout << "\nFirst character of each word and its address:\n";
    for (int i = 0; i < 3; i++) {
        cout << words[i][0] << "(" << (void*)&words[i][0] << ") ";
    }
    cout << "\n";

    return 0;
}
```